AN2DL - First Challenge Report
Team Name : Dumpling

ChengWei Hong, CaiJun Gong, XinYue Gu, Giovanni Ni

stehong77, caijungong, gretaguxinyue, giovannini

986481, 287070, 279964, 279966

November 17, 2025

1 Introduction

The project addresses a *multivariate time-series multiclass classification problem*. The objective is to develop a neural network that, given the temporal dynamics of the features *pain_survey_1–pain_survey_4*, *n_legs*, *n_hands*, *n_eyes*, and *joint_00–joint_30* for a single subject, is capable of accurately distinguishing between three levels of pain: *no pain*, *low pain*, and *high pain*.

Our approach consists of first performing an exploratory analysis of the dataset in order to understand its structure, variability, and temporal characteristics. Based on these insights, we design and adapt neural network architectures suited for sequential data.

2 Problem Analysis

The data set of the pirates with their records is composed of different features:

1. pain-survey-1 to pain-survey-4
2. n-legs, n-hands, n-eyes
3. joint-00 to joint-30

A key observation is the distinction between our two feature types: **static features** (`n_legs`, `n_hands`, and `n_eyes`), which are constant for each pirate, and **dynamic (time-varying) features** (the `joint_` and `pain_survey_` data), which change at each timestep. The model we build must recognize that the sequential data is *conditioned on* the permanent physical condition of each sample.

While the high dimensionality (large number of features) is beneficial for identifying differences between pain levels, it makes **data cleaning** essential. To this end, we worked on the correlation between different joint data to **identify and remove highly redundant features. This process reduces noise and model complexity, which helps prevent overfitting and improves training efficiency.**

Table 1: Top 3 Most Correlated Joint Pairs

column_1	**column_2**	**correlation**
joint_10	joint_11	0.949209
joint_25	joint_21	0.854405
joint_00	joint_02	0.844885

One major challenge we face is **class imbalance**. Most pirates in the dataset are labeled as *no pain*, while only a small fraction are labeled as *low pain* or *high pain*. This imbalance can significantly affect the model's performance and its ability to generalize, as the classifier may become biased toward predicting the majority class.

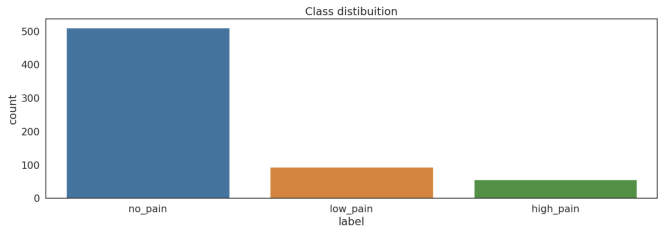

Figure 1: Class distribution

3 Method

Feature Selection

- **Variance:** Dropped `joint_30` because it had a constant value (zero variance).
- **Correlation:** Dropped `joint_11` due to its high (95%) correlation with `joint_10`, which reduced redundancy.

Preprocessing & Splitting

- Standardized all continuous features (as `float32`).
- One-hot encoded static features (`n_legs`, `n_hands`, `n_eyes`).
- Mapped pain labels to numerical values: 0 (`no_pain`), 1 (`low_pain`), 2 (`high_pain`).
- Split the 661 samples into 85% training and 15% validation sets.

Model Training & Optimization

- **Overfitting:** Addressed with Dropout, L2 regularization (λ_2).
- **Class Imbalance:** Managed by balancing the training set, using **loss weighting**, and applying oversampling/augmentation.
- **Performance:** Optimized using hyperparameter tuning (window size, stride), early stopping, and label smoothing.

4 Experiments

Our experimentation began with baseline architectures before moving to more complex sequential models. The base models that have been used are as follows:

- **Simple RNN and LSTM (Long Short-Term Memory)** We first established a baseline with a standard RNN and the unidirectional LSTM. However, it showed rapid overfitting, as expected this model can't catch the dataset details and work with complex time-series classification .
- **GRU (Gated Recurrent Unit)** We then tested a GRU model. This provided a slight improvement, was able to train for longer and getting higher scores, but this model also overfit.
- **BiGRU (Bidirectional GRU)** We hypothesized that processing the sequence in both directions would be beneficial. This was confirmed by the BiGRU model, which achieved a new best validation F1 score of 0.9280.
- **BiLSTM (Bidirectional LSTM) (Final Model)** Finally, we tested a Bidirectional LSTM. This architecture yielded the best performance of all models, achieving a peak validation F1 score of **0.9323**.

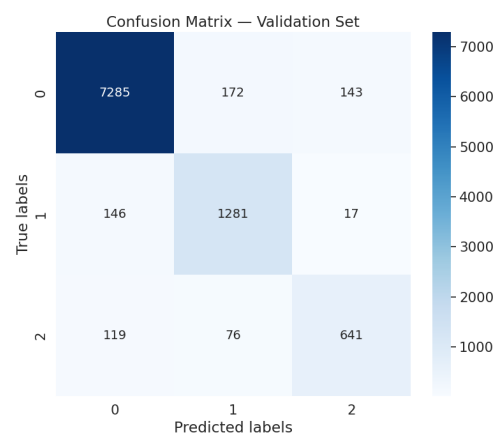

Figure 2:

- **Custom Hybrid Model (CNN-RNN)** We built a dual-stream hybrid architecture. It processes static and dynamic features in parallel before fusion.
  1. **Static Stream:** `nn.Embedding` layers learn a dense vector representation for permanent conditions (`n_legs`, `n_hands`, `n_eyes`).

Table 2: Best results of each model on the validation set.

Model	Accuracy	Precision	Recall	F1 Score
biGRU	0.9295	0.9289	0.9290	0.9280
Custom Hybrid Mode	0.8339	0.8168	0.8339	0.8098
biLSTM	**0.9438**	**0.9425**	**0.9438**	**0.9428**

2. **Dynamic Stream:** A `nn.Conv1d` (1D CNN) layer acts as a feature extractor, identifying local temporal patterns in the sequential data.
3. **Fusion & Classification:** The outputs of both streams are concatenated at each timestep. This "enriched" sequence is then fed into a recurrent layer (BiGRU/BiLSTM) for final classification.

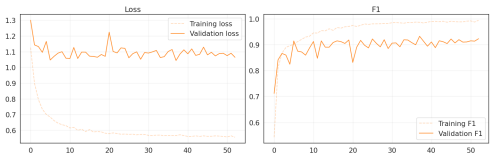

Figure 3: BILSTM training and validation F1 score and loss

5 Results

During the week, we experimented with several strategies to improve this result, such as adding a CNN layer inside the model, modifying the dropout rate, and increasing the number of hidden layers or the hidden size.

Our best-performing model is a BiLSTM with two hidden layers and a hidden size of 128, trained with class weighting to mitigate the imbalance problem, a learning rate of 0.0001, dropout of 0.4, and L2 regularization. This configuration achieved our highest score on the Kaggle test set, reaching 0.9551.

Interestingly, despite some small improvements, adding more complexity to the model (e.g., additional CNN layers or larger architectures) did not lead to substantial gains; instead, it tended to increase overfitting. This outcome is reasonable, as the dataset is not complex enough to require a highly sophisticated model. A BiLSTM without any internal CNN layers is already sufficient to capture the relevant patterns.

6 Discussion

In this section, analyse your results critically. Consider:

- Strengths and weaknesses:
  Our model shows strong generalization ability, as indicated by the consistent validation scores across different folds. However, the model still struggles a little bit with classes that have fewer samples, suggesting some sensitivity to class imbalance.
- Limitations and assumptions:
  The dataset suffers from class imbalance, which can negatively affect the model's ability to learn minority classes and may introduce bias in the predictions. The model relies on the assumption that the dataset contains real and noise-free samples, reflecting ideal acquisition conditions.

7 Conclusions

- Restate main contributions:
  Every teammate gave their best effort, and everyone worked hard to achieve the best possible results. Each teammate contributed in different ways during the various phases of the project.
- Suggest improvements:
  To perform a more in-depth analysis of the dataset in order to extract additional useful information that can help improve the construction and training of the model.
- Propose future work:
  One possible direction for future work is to perform a deeper analysis of the dataset to extract additional insights that could be useful for the model